

# Android app development, made understandable.

A visual first step into Android Studio, Kotlin, and useful mobile apps.

**Beginner promise:** start with a problem, build one small thing, and learn by testing it.



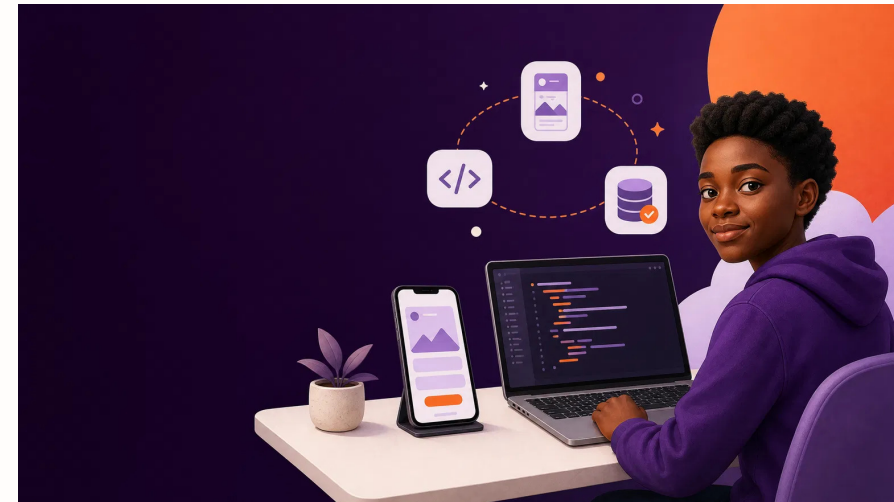
# Start with one useful problem.

A good first app is not huge. It helps one person complete one job more easily.

---

**Our class project: Campus Tasks** — capture a task, mark it done, and find it later.

Ask before you code: “Who needs this?”



---

## Screen

What a person sees and touches.

---

## Logic

The rules that decide what happens.

---

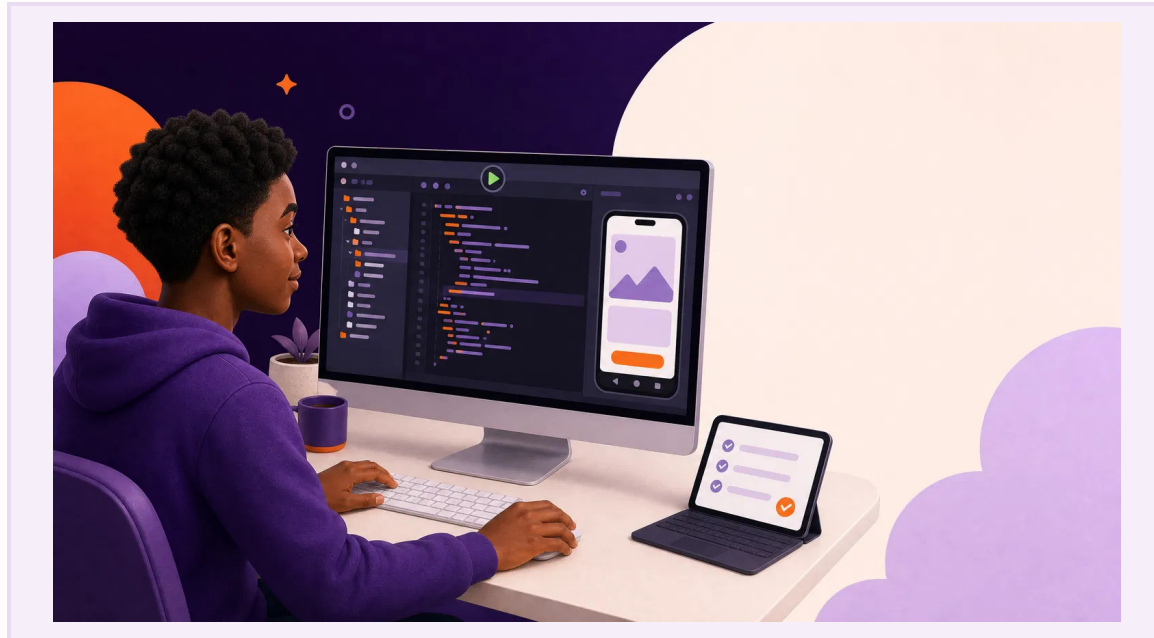
## Data

What the app remembers.

---

**Quick prompt:** What is one daily student problem a phone could reduce?

# Android Studio is your workshop.



- Create the project, write code, preview the app, and fix mistakes in one place.
- The **emulator** is a practice phone inside your computer.
- Press **Run** to send your app to the emulator.

## TRY IT

Find the project panel, editor, Run button, emulator, and Logcat before writing code.

# Kotlin gives precise instructions.

## Variable

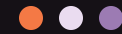
Remembers a value, such as a task title or completion status.

## Function

Groups one action, such as add a task or clear the input.

## Class

Describes a useful thing, such as a `Task`.



```
data class Task(  
    val title: String,  
    val done: Boolean = false  
)
```

---

### Read it aloud:

*“A task has a title and begins unfinished.”*

# Compose turns code into a screen.

- A **composable** is a reusable piece of interface: title, button, card, or task row.
- Describe what you want to see; Compose draws it for you.
- Keep each composable focused on one small job.

---

**Try it:** Change the sample title, run the app, and explain what changed.

## KOTLIN CODE

```
@Composable
fun TaskRow(task: Task) {
    Text(text = task.title)
}
```



## SCREEN OUTPUT

The screen output shows a single task row. It consists of a light purple vertical bar on the left, followed by a white circle, and then the text 'Buy groceries' in a dark purple font. The entire row is set against a light purple background.

○ Buy groceries

# State explains screen changes.

- **State** is the app's current memory: input text, selected item, task list, or loading status.
- A tap creates an event; the event changes state; the screen redraws.
- Keep one clear source of truth for each important piece of state.

---

**Pair activity:** Act out the loop: tap → event → state → screen.

```
var taskName by remember {  
    mutableStateOf("")  
}
```

## THE UNIDIRECTIONAL FLOW



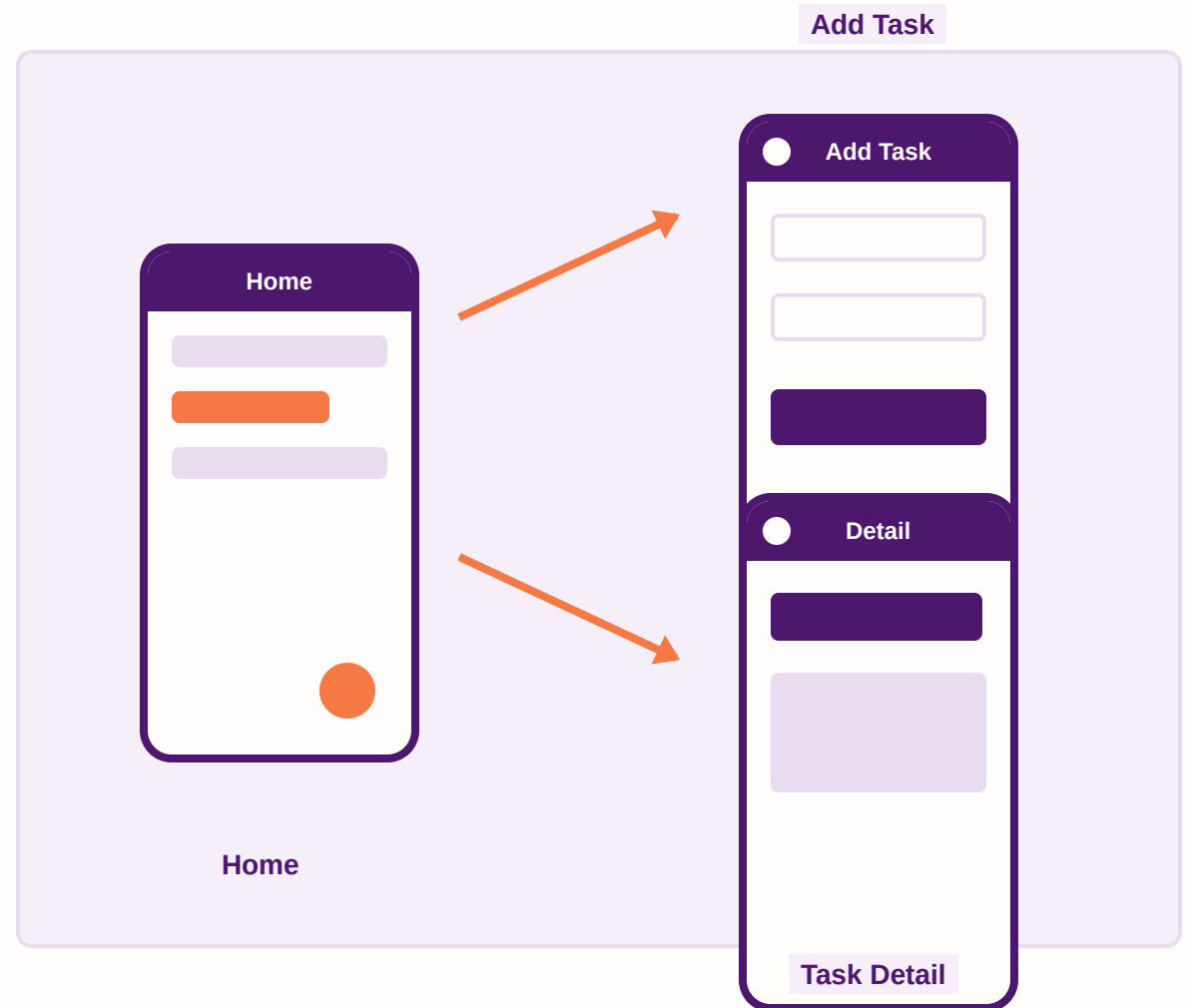
# A small screen map prevents confusion.

- Sketch the journey before coding: **Home** → **Add task** → **Task detail**.
- Give every screen one purpose and one obvious next action.
- Always provide a clear route back.

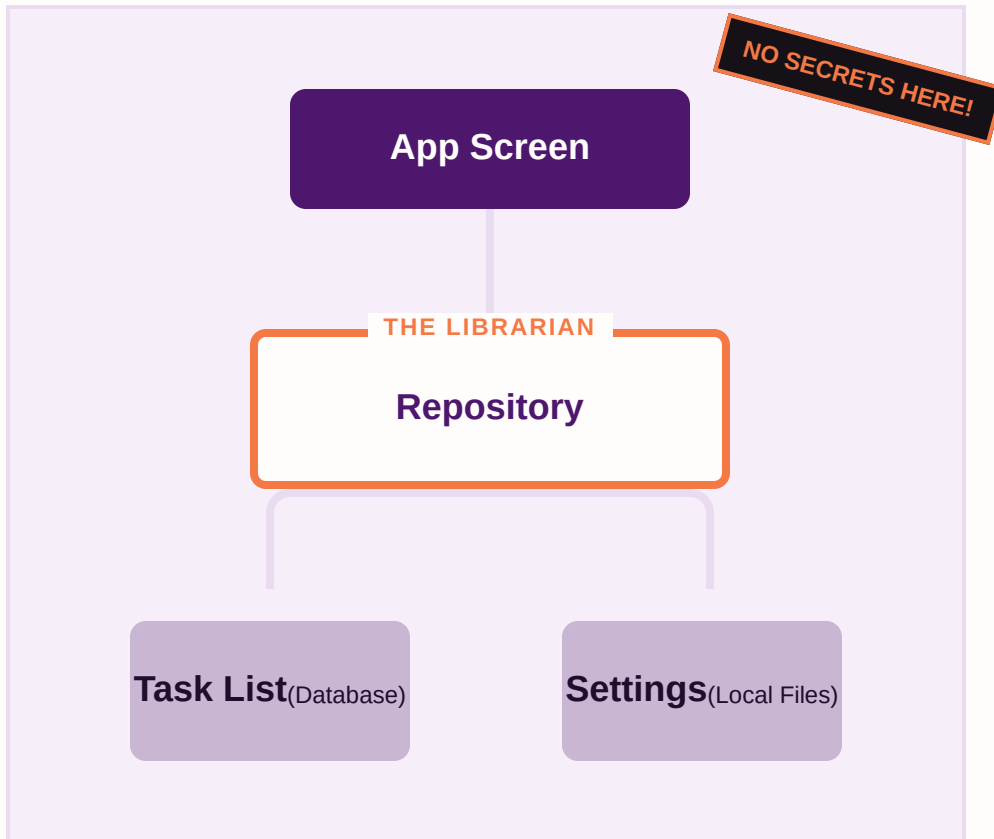
---

## Design question:

What information must travel from Home to Task detail?



# Good apps remember safely.



- Save simple settings locally; use structured storage for a growing task list.
- A **repository** gives the app one dependable place to request and update data.
- Never place passwords, API keys, or private secrets in the app code.

---

## Memory aid:

A repository is a librarian: it knows where the information belongs.



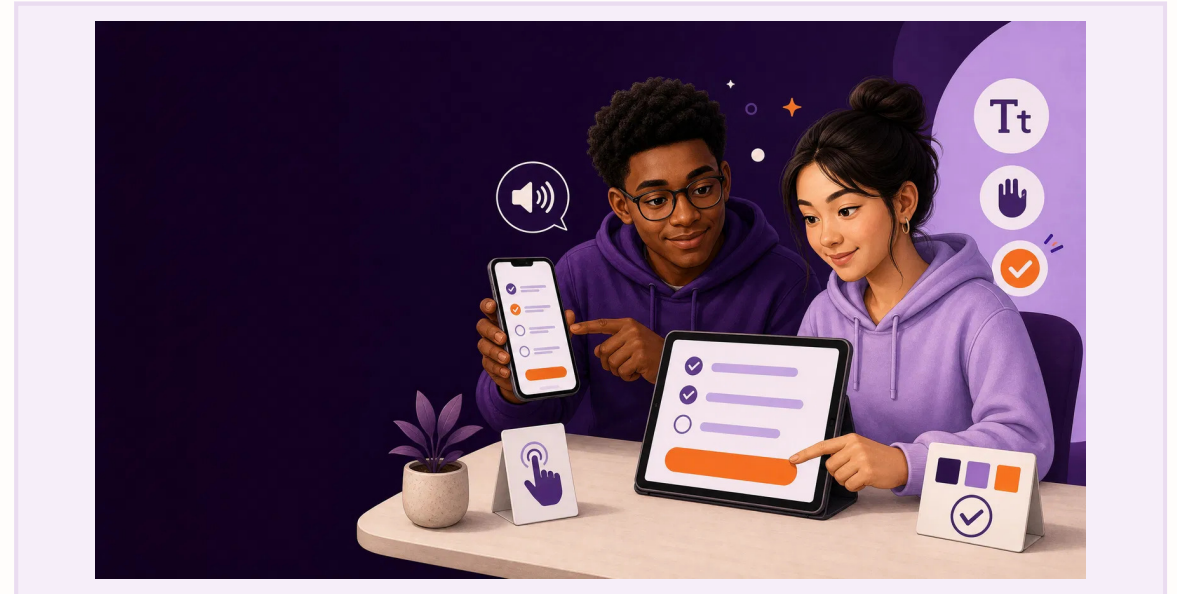
# Build for more than one kind of user.

- Use clear labels, readable contrast, and buttons that are easy to touch.
- Let text grow without breaking the layout.
- Test important actions with accessibility tools and real people.

---

## Accessibility check:

Can someone understand the task button without seeing its colour?



# Test before you trust it.

---

01

## Small Rules

Test the small rules first:  
*“Does a new task begin  
unfinished?”*

---

02

## Full Journeys

Test the full journey on an  
emulator or a real phone.

---

03

## Failure Cases

Test failures too: blank input,  
no connection, slow loading,  
and rotation.

---

### Team challenge:

Write one failure case your classmates should try today.

# Build a small app, then improve it.

1

## WEEK 1

Plan the learner, problem, and three screens.

2

## WEEK 2

Build a working screen, button, and simple task row.

3

## WEEK 3

Add state, a list, and navigation.

4

## WEEK 4

Save data, test, improve accessibility, and present your work.

---

FINISH LINE

A useful Campus Tasks app another student can actually test.

# Keep learning with trusted guides.

---

## OFFICIAL LEARNING

### Android Basics with Compose

Google's official beginner curriculum.

[developer.android.com/courses](https://developer.android.com/courses)

### Jetpack Compose Docs

The complete guide to building UI.

[developer.android.com/develop/ui/compose](https://developer.android.com/develop/ui/compose)

---

## ACCESSIBILITY & TESTING

### Android Accessibility

Rules for building inclusive applications.

[developer.android.com/guide/topics/ui/accessibility](https://developer.android.com/guide/topics/ui/accessibility)

### App Architecture

How to separate screens from data safely.

[developer.android.com/topic/architecture](https://developer.android.com/topic/architecture)

---

## YOUR NEXT STEP

Open Android Studio and start your first project.